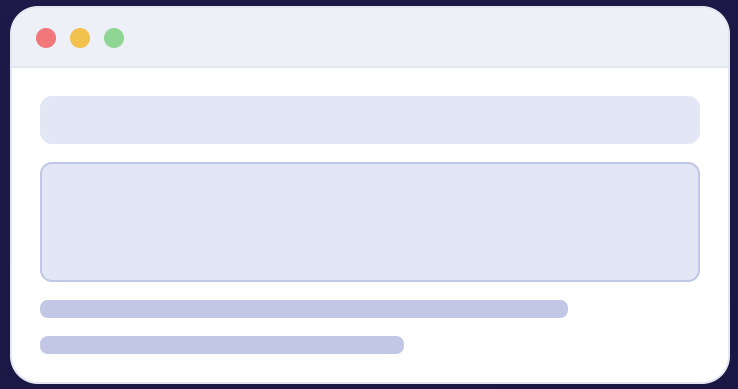
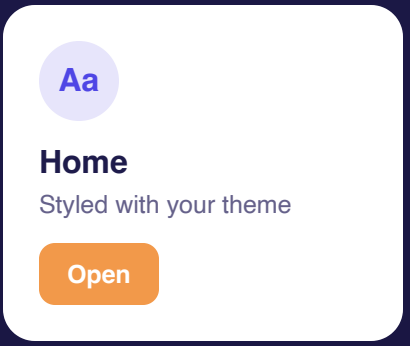
6APSI · REACT FINAL PROJECT PLANNING

Design Systems

Turning your wireframes into a real React theme - from greyscale boxes to a styled app.

Where we're headed: your palette, type, and components become your theme - CSS variables, a Tailwind config, or a `ThemeProvider` - on day one of the build.





THE JOURNEY

Three docs, one React app

DOC 2 - DONE

Wireframe

Greyscale boxes. Screens, flow, and a component tree, mapped on paper.

DOC 3 - TODAY

Design system

The tokens and parts that make your app look intentional.

THE BUILD

React project

Your styled, working app - built straight from these two documents.

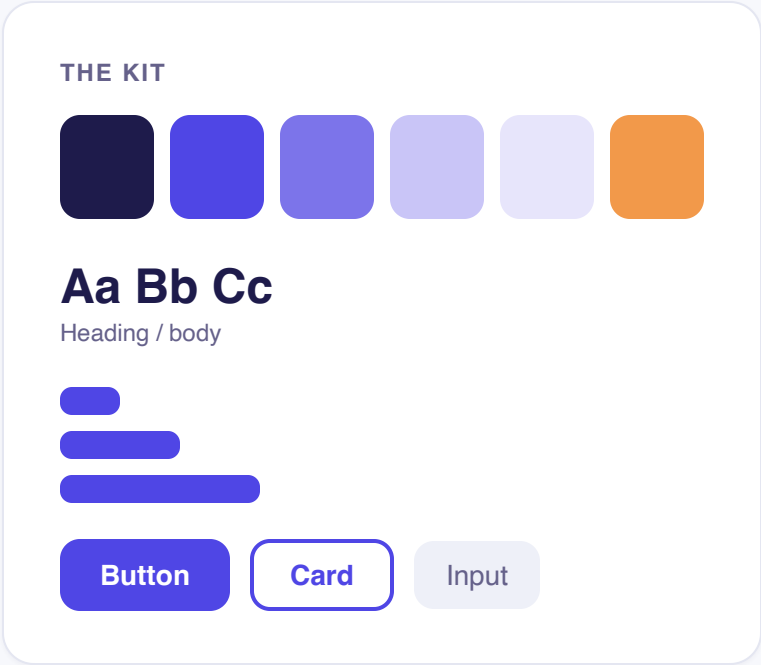
This deck bridges your wireframes (Doc 2) and the first day you run `npm run dev`.

DEFINITION

What's a design system?

A reusable kit of decisions and parts - colors, type, spacing, and components - plus the rules for using them, so every screen looks like it belongs to the same app and you build faster.

Think of a LEGO set. You don't carve each brick from scratch - you snap together pieces made to fit. Same colors, same sizes, every time.



Sources: Nielsen Norman Group - Design Systems 101; Brad Frost - Atomic Design. Ref: react-theory/09 - Design systems

WHY BOTHER

Why use a system?

Consistent

Define a <Button> once; it looks the same on every screen. No drift.

Fast

Reuse components instead of rebuilding them. Style three screens in the time of one.

Professional

Cohesive color, type, and spacing is what makes an app look "designed."

Easy to change

Change one token - say your accent color - and it updates everywhere.

Sources: Nielsen Norman Group - Design Systems 101; Contentful - Design Tokens

HOW IT'S BUILT UP

Small parts → whole screens

Brad Frost's "atomic design" (2013) maps almost perfectly onto React's component tree: build from the smallest pieces up. Change a small part and everything using it updates.

Tokens

Raw decisions: a color, a size, a space

Atoms

Smallest UI: a `Button`, `Input`

Molecules

A few atoms joined: a `Card`

Organisms

A section: a `Header`, a card grid

Pages

Real content in the layout: your app

Don't sweat the labels. Frost himself says they're just a mental model - the point is: nest small parts into bigger ones. They map to your `src/components/` folders.

Sources: Brad Frost - Atomic Design (atomicdesign.bradfrost.com). Ref: react-theory/08 - Atomic design

THE THREE YOU'LL SET

Design tokens = saved decisions

A token is a design decision saved as a reusable named value. In React they live in your theme - and **where** depends on your styling stack:

CSS Modules / CSS

`:root` custom properties: `--color-primary`

Tailwind

the `theme` block in `tailwind.config.js`

styled-components

a `theme` object via `<ThemeProvider>`

UI library (MUI, Chakra)

the library's theme object - override its defaults

Meta moment: this very deck runs on a tiny design system too - one indigo, one amber, neutral greys, two type sizes, and 8-pt spacing.

Sources: MDN - Using CSS custom properties; Design Tokens Community Group (W3C). Ref: m3-styling/07 - Choosing your stack

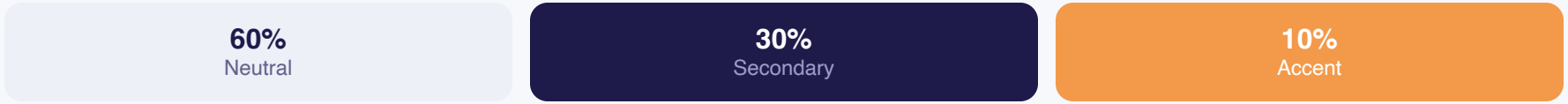
TOKEN 1 - COLOR

Pick a palette, then ration it

Your palette = one brand color + neutrals + one accent. A brand color isn't one value - it's a ramp of tints & shades:

As tokens: 500 → `--color-primary`, 900 → `--color-text`, 50 → `--color-surface`.

The 60-30-10 rule keeps it balanced:



APPLIED TO A CARD

Project card

Neutral surface (60%) lets dark text (30%) read clearly; one accent (10%) pulls the eye to the action.

View

Sources: 60-30-10 rule - LogRocket; freeCodeCamp; Material Design 3 - Color System

COLOR, PART 2

Make sure people can read it

Pretty doesn't matter if it's unreadable. Aim for a contrast ratio of at least **4.5 : 1** between text and its background (WCAG AA). This is on the grading rubric.

Sample
body text
FAILS
Light grey on white

Sample
body text
FAILS
Amber text on white

Sample
body text
PASSES
Dark ink on white

Sample
body text
PASSES
White on indigo

Tip: run each text-on-background pair through the WebAIM Contrast Checker before you commit to the palette - it takes 30 seconds and saves a rubric point.

Sources: WCAG AA 4.5:1 - via WebAIM Contrast Checker

TOKEN 2 - TYPE

Two fonts, a few sizes

Limit yourself to **1–2 typefaces** (a heading + a body, or one family in different weights), sized on a **consistent scale** - not random numbers.

Same rule as your wireframes: 2–3 sizes build hierarchy. A type scale just makes those sizes feel related.

Tip: pairing one family in Regular + Bold is the foolproof option.

YOUR TYPE TOKENS

Heading
`--font-size-xl · text-3xl`

Subhead
`--font-size-lg · text-xl`

Body 16
`--font-size-base · text-base`

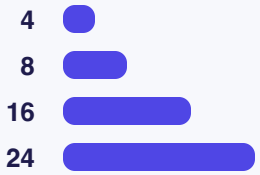
Caption 13
`--font-size-sm · text-sm`

Sources: Material Design 3 - Type Scale (m3.material.io); ATNO - Typography in UI Design

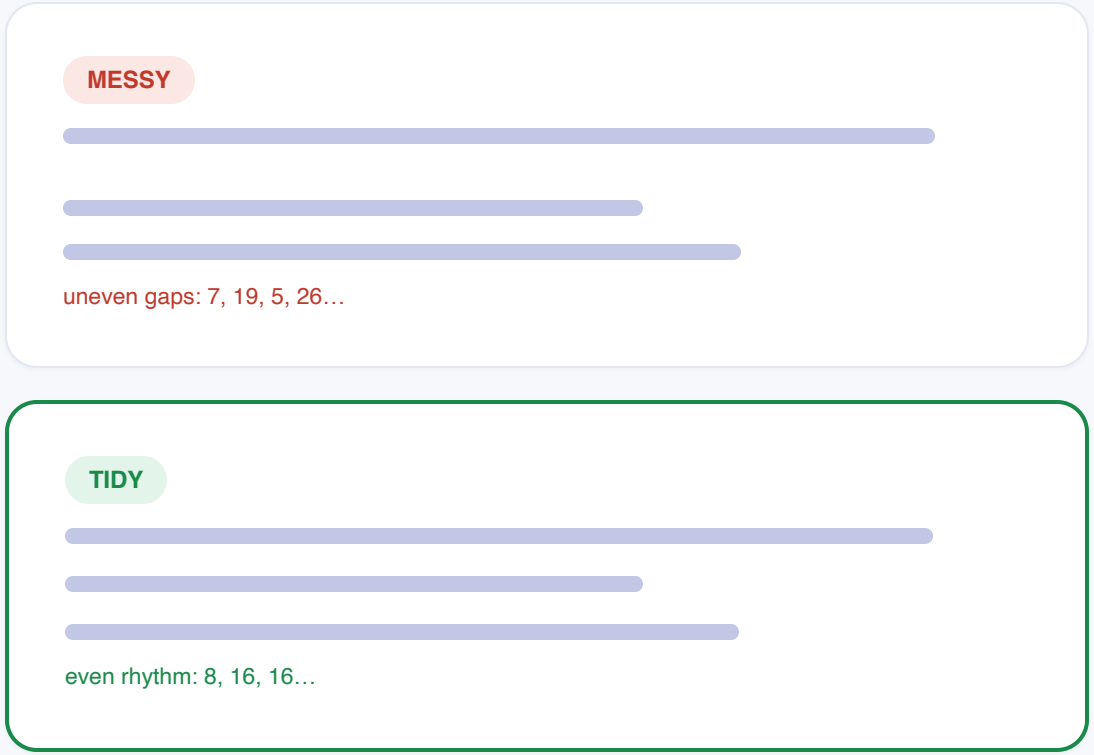
TOKEN 3 - SPACING

Space on a system, not by feel

Pick a spacing scale and stick to it - **multiples of 8** (with 4 for tight spots). Every gap, padding, and margin comes from the scale.



In CSS: `gap: var(--space-4)`, `padding: 16px`. In Tailwind: `gap-4`, `p-6` - always a step on the scale.



Sources: Material Design 3 - Spacing; U.S. Web Design System - Spacing Tokens

COMPONENTS - THE BRIDGE

Turn each box into a real component

Every greyscale box from your wireframes becomes a styled React component, built from your tokens.
Same structure - now with color, type, and spacing applied.

WIREFRAME

STYLED

Get started

→ <Button>

WIREFRAME

STYLED

Project
short caption

→ <Card>

WIREFRAME

STYLED

Logo Home About

→ <Header>

Source: React documentation - Thinking in React; MDN - HTML landmark elements

ONE MORE THING ABOUT PARTS

Components have states

On the web the same button looks different depending on what the user is doing. Plan these now - they make your build feel real, and they are just CSS pseudo-classes plus a little state.

Default

resting state

Hover / active

`:hover · :active`

Disabled

`disabled prop`

Loading

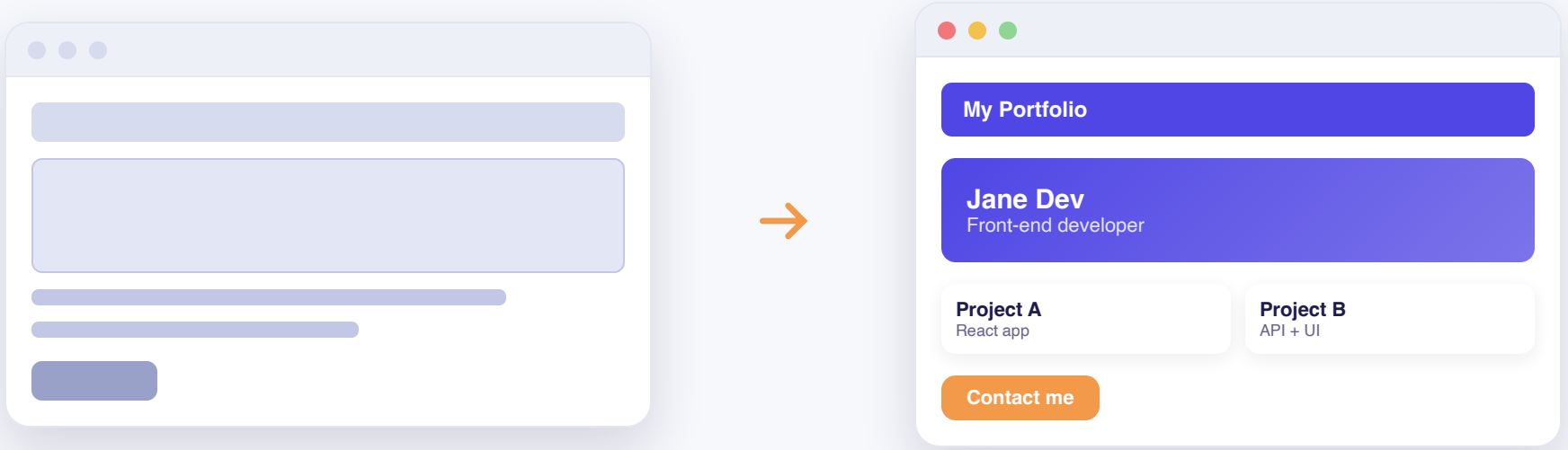
waiting on data

Simple recipe: hover/active = a bit darker (`:hover`), disabled = greyed and inert (the `disabled` prop), loading = a spinner via conditional rendering on a `useState` flag.

Sources: MDN - CSS pseudo-classes; React documentation - Conditional rendering & state

THE PAYOFF

Same screen, leveled up



Same wireframe + tokens (color, type, spacing) + components = a screen that feels finished. The structure never changed - only the design system did.

Same layout, same content structure - only the design system changed.

YOU DON'T START FROM ZERO

Borrow from the pros

React has a rich ecosystem of ready-made design systems. Adopt one for accessible, battle-tested components, or study them and build your own theme - and sketch it in Figma before you touch code.

UI libraries

MUI, Chakra UI, or shadcn/ui + Radix give you a themeable, accessible component set out of the box.

Storybook

The standard way to build and document each component in isolation - a living catalog of your system.

Figma

Sketch your palette, type, and components visually before you commit them to code.

Borrowing structure is normal and smart - just make the colors and content your own.

Sources: MUI; Chakra UI; shadcn/ui; Storybook; Figma. Ref: m3-styling/06 - UI component libraries

BEFORE THE REACT BUILD

Fill in your design system today

1. **Pick your stack**
   CSS Modules · Tailwind · styled-components · or a UI library
2. **Choose your tokens**
   1 brand color + neutrals + 1 accent · 1–2 fonts · an 8-pt spacing scale
3. **List your components**
   Turn every wireframe box into a named component (Button, Card, Header)
4. **Check contrast**
   Body text at least 4.5:1 - run it through WebAIM

Avoid these

- Too many colors or fonts
- Every button styled differently
- Spacing chosen by feel
- Low-contrast text
- All-inline styles (no real approach)

Deliverable: your filled-in Design System doc (Doc 3) - open it every time you build a new component.

REFERENCES

Sources

- **Brad Frost** - Atomic Design (atomicdesign.bradfrost.com): tokens → atoms → molecules → organisms → pages
- **Nielsen Norman Group** - "Design Systems 101" (nngroup.com)
- **Google - Material Design 3** - Design Tokens, Color System, Type Scale, Spacing (m3.material.io)
- **React documentation** - "Thinking in React"; conditional rendering & state (react.dev)
- **MDN Web Docs** - Using CSS custom properties; CSS pseudo-classes; HTML landmarks (developer.mozilla.org)
- **Design Tokens Community Group (W3C)** - design tokens spec (designtokens.org)
- **U.S. Web Design System** - Design Tokens & Spacing (designsystem.digital.gov)
- **MUI, Chakra UI, shadcn/ui, Radix** - React component libraries
- **Storybook** - building & documenting components in isolation (storybook.js.org)
- **60-30-10 rule** - LogRocket; freeCodeCamp · **Contrast** - WCAG AA 4.5:1 via WebAIM
- **Figma** - design & component prototyping